

# Дополнительные идиомы и механизмы C++

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

**Александра Волосова,**  
к.т.н., доцент кафедры  
**ИУ5**

# Часть 1

**Variadic шаблоны и свёртка**

# Метапрограммирование:

- **Константы времени компиляции** — основа метапрограммирования
- **Variadic шаблоны, свёртка** — инструмент метапрограммирования для работы с произвольным числом типов/значений
- **SFINAE, enable\_if** — классическая техника *условной*\* компиляции шаблонов

\* При обнаружении ошибки, компилятор игнорирует неподходящего кандидата

- **Variadic шаблоны** позволяют написать один шаблон, который принимает **любое количество** аргументов (типов или значений)
- **Свёртка (fold)** — спос об **свернуть** все эти аргументы в одно выражение с помощью оператора (например, +, &&, ,)

**// Без variadic необходимо много перегрузок**

```
void print() {}
```

```
void print(int a) {}
```

```
void print(int a, int b) {}
```

```
void print(int a, int b, int c) {}
```

**// С использованием variadic нужна только одна функция**

```
template<typename... Args>
```

```
void print(Args... args) {
```

```
    (std::cout << ... << args); // свёртка
```

```
}
```

// Без свёртки - рекурсия

```
template<typename T>
```

```
T sum(T t) { return t; }
```

```
template<typename T, typename... Rest>
```

```
T sum(T first, Rest... rest) {
```

```
    return first + sum(rest...);
```

```
}
```

// **Со свёрткой**

```
template<typename... Args>
```

```
auto sum(Args... args) {
```

```
    return (args + ...);}
```

// Вызов с 1 аргументом  
 $\text{sum}(5) \rightarrow (5 + \dots) \rightarrow 5$

// Вызов с 2 аргументами  
 $\text{sum}(3, 4) \rightarrow (3 + 4) \rightarrow 7$

// Вызов с 3 аргументами  
 $\text{sum}(1, 2, 3) \rightarrow (1 + (2 + 3)) \rightarrow 6$

**Задача: написать функцию, которая принимает  
любое количество строк и выполняет их  
конкатенацию через запятую**

**Вход: "a", "b", "c "**

**Выход: "a, b, c"**

# Variadic шаблоны, свёртка

## Обычное решение

```
#include <iostream>
#include <string>
using namespace std;
template<typename T>
string join(const T& arg) {
    return string(arg);
}
template<typename T, typename... Args>
string join(const T& first, const Args&... rest) {
    return string(first) + ", " + join(rest...);
}
int main() {
    cout << join("a", "b", "c") << endl;    // a, b, c
    cout << join("Hello", "World", "from", "C++") <<
endl; // Hello, World, from, C++
    return 0;
}
```

### Your Output

```
a, b, c
Hello, World, from, C++
```



# Решение с variadic и свёрткой

```
#include <iostream>
#include <string>
using namespace std;
// Функция принимает любое количество строк и выполняет их конкатенацию через запятую
template<typename... Strings>
string join(Strings... strs) {
    string result;
    bool first = true;
    ((result += (first ? (first = false, "") : ", ") + string(strs)), ...);
    return result;
}

int main() {
    // Тестовые примеры
    cout << join("a", "b", "c") << endl;      // a, b, c
    cout << join("Hello", "World") << endl;    // Hello, World
    cout << join("just one") << endl;          // just one
    cout << join("C++", "variadic", "templates", "fold") << endl; // C++, variadic, templates, fold
    return 0;
}
```

## Your Output

```
a, b, c
Hello, World
just one
C++, variadic, templates, fold
```

# Использование variadic и свёртки

## 1. Логирование и отладка

// Логгер с любым количеством аргументов

```
#include <iostream>
```

```
template<typename... Args>  
void debug(Args&&... args) {  
    std::cout << "[DEBUG] ";  
    ((std::cout << args << " "), ...);  
    std::cout << std::endl;  
}
```

```
int main() {  
    debug("Player:", 10, 20, "HP:", 85);  
    debug("File:", "config.json", "size:", 1024);  
}
```

### Your Output

```
[DEBUG] Player: 10 20 HP: 85
```

```
[DEBUG] File: config.json size: 1024
```

# Использование variadic и свёртки

## 2. Проверка условий в тестах

```
#include <iostream>
#include <cassert>
template<typename... Args>
bool expectAll(Args... args) {
    return (args && ...);
}
// Структура для тестирования
struct User {
    bool isValid() const { return true; }
    bool hasSubscription() const { return true; }
    bool isActive() const { return true; }
    int age = 25;
};
```

### Your Output

```
Test 1 passed: All conditions true
Test 2 passed: Age condition failed
```

```
int main() {
    User user;
    // Тест 1: Все условия выполняются
    assert(expectAll(
        user.isValid(),
        user.age > 18,
        user.hasSubscription(),
        user.isActive()
    ));
    std::cout << "Test 1 passed: All conditions true\n";
    // Тест 2: Одно условие не выполняется
    user.age = 16;
    if (!expectAll(
        user.isValid(),
        user.age > 18,
        user.hasSubscription(),
        user.isActive()
    )) {
        std::cout << "Test 2 passed: Age condition failed\n";
    }
    return 0;
}
```

# Использование variadic и свёртки

## 3. Сборка и передача параметров в конструкторы

```
#include <iostream>
#include <memory>
#include <string>
// Эмуляция emplace для своих классов
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>
        (args)...));
}
// Класс Player
enum class Team { Red, Blue, Green };
class Player {
    std::string name;
    int health;
    float speed;
    Team team;
public:
    Player(const std::string& n, int h, float s, Team t)
        : name(n), health(h), speed(s), team(t) {}
};
```

### Your Output

```
Name: John, HP: 100, Speed: 50.5, Team: Red
Name: Alice, HP: 85, Speed: 62.3, Team: Blue
Name: Bob, HP: 120, Speed: 45, Team: Green
```

```
void print() const {
    std::cout << "Name: " << name
        << ", HP: " << health
        << ", Speed: " << speed
        << ", Team: " << (team == Team::Red ? "Red" :
            team == Team::Blue ? "Blue" : "Green") <<
        "\n";
}
};
int main() {
    auto player = make_unique<Player>("John", 100, 50.5f,
        Team::Red);
    player->print();

    auto player2 = make_unique<Player>("Alice", 85, 62.3f,
        Team::Blue);
    player2->print();

    auto player3 = make_unique<Player>("Bob", 120, 45.0f,
        Team::Green);
    player3->print();

    return 0;
}
```

# Использование variadic и свёртки

## 4. Создание SQL запросов

```
#include <iostream>

#include <string>

// Построение WHERE условия с параметрами
template<typename... Conditions>
std::string buildWhere(Conditions... conds) {
    std::string result = "WHERE ";
    ((result += std::string(conds) + " AND "), ...);
    return result.substr(0, result.length() - 5); }
```

### Your Output

```
WHERE age > 18 AND status = 'active' AND country = 'RU'
WHERE price < 100 AND in_stock = true
WHERE user_id = 123
WHERE name = 'Alice' AND age = 25
```

```
int main() {
    auto query1 = buildWhere("age > 18", "status =
'active'", "country = 'RU'");
    std::cout << query1 << std::endl;
    auto query2 = buildWhere("price < 100",
"in_stock = true");
    std::cout << query2 << std::endl;
    auto query3 = buildWhere("user_id = 123");
    std::cout << query3 << std::endl;
    // std::string
    std::string condition = "name = 'Alice'";
    auto query4 = buildWhere(condition, "age =
25");
    std::cout << query4 << std::endl;
    return 0;
}
```

# Использование variadic и свёртки

## 5. Обработка событий

```
#include <iostream>
#include <string>
// Вызов нескольких обработчиков события
template<typename... Handlers>
void notifyAll(Handlers&&... handlers) {
    (handlers(), ...); // Вызвать каждый обработчик
}
// Примеры функций-обработчиков
void saveToDatabase() {
    std::cout << "Saving to database..." << std::endl;
}
void updateCache() {
    std::cout << "Updating cache..." << std::endl;
}
void sendWebhook() {
    std::cout << "Sending webhook..." << std::endl;
}
void logEvent() {
    std::cout << "Logging event..." << std::endl;
}
// Функция с параметром
void sendEmail(const std::string& email) {
    std::cout << "Sending email to " << email << std::endl;
}
```

```
int main() {
    // Пример 1: Функции без параметров
    std::cout << "=== Event: User Registered ===" << std::endl;
    notifyAll(
        saveToDatabase,
        updateCache,
        sendWebhook,
        logEvent
    );
    // Пример 2: Лямбда-обработчики
    std::cout << "\n=== Event: Order Placed ===" << std::endl;
    std::string orderId = "ORD-12345";
    int total = 150;
    notifyAll(
        []() { std::cout << "Validate order" << std::endl; },
        []() { std::cout << "Process payment" << std::endl; },
        []() { std::cout << "Update inventory" << std::endl; },
        []() { std::cout << "Send confirmation" << std::endl; }
    );
}
```

# Использование variadic и свёртки

## 5. Обработка событий

// Пример 3: Смешанные обработчики (функции + лямбды)

```
std::cout << "\n=== Event: Application Started ===" << std::endl;
```

```
notifyAll(
```

```
    []() { std::cout << "Load configuration" << std::endl; },
```

```
    []() { std::cout << "Initialize logging" << std::endl; },
```

```
    []() { std::cout << "Connect to database" << std::endl; },
```

```
    []() { std::cout << "Start web server" << std::endl; }
```

```
);
```

// Пример 4: Обработчики с захватом переменных

```
std::cout << "\n=== Event: Data Updated ===" << std::endl;
```

```
int recordCount = 42;
```

```
notifyAll(
```

```
    [&]() { std::cout << "Updated " << recordCount << " records" << std::endl; },
```

```
    [&]() { std::cout << "Refresh UI with new data" << std::endl; },
```

```
    [&]() { std::cout << "Notify subscribers" << std::endl; }
```

```
);
```

```
return 0;
```

```
}
```

### Your Output

```
=== Event: User Registered ===  
Saving to database...  
Updating cache...  
Sending webhook...  
Logging event...
```

```
=== Event: Order Placed ===  
Validate order  
Process payment  
Update inventory  
Send confirmation
```

```
=== Event: Application Started ===  
Load configuration  
Initialize logging  
Connect to database  
Start web server
```

```
=== Event: Data Updated ===  
Updated 42 records  
Refresh UI with new data  
Notify subscribers
```

# Использование variadic и свёртки

## 6. Валидация формы

```
#include <iostream>
#include <string>
// Проверка всех полей формы
template<typename... Validators>
bool validate(Validators... validators) {
    return (validators && ...);
}

void printForm(const std::string& name, const
std::string& email, const std::string&
password) {
    std::cout << "\n-Form Data-" << std::endl;
    std::cout << "Name: " << name << std::endl;
    std::cout << "Email: " << email << std::endl;
    std::cout << "Password: " <<
std::string(password.length(), '*') << " (" <<
password.length() << " chars)" << std::endl;
}
```

```
void printValidation(bool isValid) {
    std::cout << "Form valid: " << std::boolalpha << isValid <<
std::endl;
    if (isValid) {
        std::cout << "✓ Form submitted successfully!" << std::endl;
    } else {
        std::cout << "✗ Please fix the errors above" << std::endl;
    }
}
```



# Использование variadic и свёртки

## 6. Валидация формы

```
int main() {  
    // Пример 1: Все поля валидны  
    std::cout << "\n-TEST 1: VALID FORM-";  
    std::string name1 = "John Doe";  
    std::string email1 = "john@example.com";  
    std::string password1 = "secure123";  
    printForm(name1, email1, password1);  
    bool isValid1 = validate(  
        !name1.empty(),  
        email1.find('@') != std::string::npos,  
        password1.length() >= 8  
    );  
    printValidation(isValid1);  
    // Пример 2: Ошибка валидации  
    std::cout << "\n-TEST 2: INVALID FORM-";  
    std::string name2 = "Alice";  
    std::string email2 = "invalid-email";  
    std::string password2 = "123";  
    printForm(name2, email2, password2);
```

```
    bool cond1 = !name2.empty();  
    bool cond2 = email2.find('@') != std::string::npos;  
    bool cond3 = password2.length() >= 8;  
    std::cout << "\nValidation details:" << std::endl;  
    std::cout << " - Name not empty: " << std::boolalpha << cond1 << std::endl;  
    std::cout << " - Valid email: " << cond2 << std::endl;  
    std::cout << " - Password (min 8 chars): " << cond3 << std::endl;  
    bool isValid2 = validate(cond1, cond2, cond3);  
    printValidation(isValid2);  
    return 0;  
}
```

```
-TEST 2: INVALID FORM-  
-Form Data-  
Name: Alice  
Email: invalid-email  
Password: *** (3 chars)
```

```
-TEST 1: VALID FORM-  
-Form Data-  
Name: John Doe  
Email: john@example.com  
Password: ***** (9 chars)  
Form valid: true  
Form submitted successfully!
```

```
Validation details:  
 - Name not empty: true  
 - Valid email: false  
 - Password (min 8 chars): false  
Form valid: false  
Please fix the errors above
```

# Использование variadic и свёртки

## 7. Сериализация данных в JSON

```
#include <iostream>
#include <string>
// Сборка JSON строки из полей
template<typename... Pairs>
std::string toJson(Pairs&&... pairs) {
    std::string json = "{";
    ((json += std::string(std::forward<Pairs>(pairs)) + ","), ...);
    if (json.back() == ',') json.pop_back();
    json += "}";
    return json;
}
```

### Your Output

```
{"name":"Alice","age":30,"city":"NYC"}
{"id":101,"name":"Bob","active":true,"score":95.5}
{"status":"ok"}
```

```
int main() {
    // Пример 1: Простой JSON
    std::string user1 = toJson(
        "\"name\": \"Alice\"", "\"age\": 30",
        "\"city\": \"NYC\""
    );
    std::cout << user1 << std::endl;
    // Пример 2: JSON с разными типами
    std::string user2 = toJson(
        "\"id\": 101", "\"name\": \"Bob\"",
        "\"active\": true", "\"score\": 95.5"
    );
    std::cout << user2 << std::endl;
    // Пример 3: Один параметр
    std::string single = toJson("\"status\": \"ok\"");
    std::cout << single << std::endl;
    return 0;
}
```

# Использование variadic и свёртки

## 8. Измерение времени выполнения

```
#include <iostream>
#include <chrono>
#include <vector>
#include <algorithm>
// Измерение времени выполнения функции
template<typename Func>
double measureTime(Func&& func) {
    auto start = std::chrono::high_resolution_clock::now();
    func();
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> duration = end
- start;
    return duration.count();
}
```

```
// Бенчмарк нескольких функций
template<typename... Funcs>
void benchmarkAll(Funcs&&... funcs) {
    ((std::cout <<
measureTime(std::forward<Funcs>(funcs)) << "
ms\n"), ...);
}
// Примеры алгоритмов сортировки
void quickSort(std::vector<int>& data) {
    std::sort(data.begin(), data.end());
}
void mergeSort(std::vector<int>& data) {
    std::stable_sort(data.begin(), data.end());
}
void heapSort(std::vector<int>& data) {
    std::make_heap(data.begin(), data.end());
    std::sort_heap(data.begin(), data.end());
}
```

# Использование variadic и свёртки

## 8. Измерение времени выполнения

```
int main() {  
    // Подготовка данных  
    std::vector<int> data1(1000000);  
    std::vector<int> data2(1000000);  
    std::vector<int> data3(1000000);  
    srand(time(0));  
    for (size_t i = 0; i < data1.size(); i++) {  
        int val = rand();  
        data1[i] = val;  
        data2[i] = val;  
        data3[i] = val;  
    }  
}
```

```
std::cout << "Benchmarking sort algorithms on 1,000,000  
elements:\n";  
    benchmarkAll(  
        [&]() { quickSort(data1); },  
        [&]() { mergeSort(data2); },  
        [&]() { heapSort(data3); }  
    );  
    return 0;  
}
```

### Your Output

```
Benchmarking sort algorithms on 1,000,000 elements:  
71.7825 ms  
87.4727 ms  
126.306 ms
```

# Использование variadic и свёртки

## 9. Форматирование вывода

```
#include <iostream>
#include <iomanip>
#include <string>
#include <tuple>
// Таблица с фиксированной шириной
template<typename... Columns>
void printTable(int width, Columns... cols) {
    std::cout << "|";
    ((std::cout << std::setw(width) << cols << "|"), ...);
    std::cout << std::endl;
}
```

```
template<typename... Widths, typename... Columns>
void printTableFlex(const std::tuple<Widths...>& widths, Columns...
cols) {
    static_assert(sizeof...(Widths) == sizeof...(Columns),
        "Widths and columns count must match");
    printTableFlexImpl(widths, std::index_sequence_for<Widths...>{},
cols...);
}
int main() {
    std::cout << "\nТаблица с фиксированной шириной 15 \n";
    printTable(15, "Name", "Score", "Time", "Status");
    printTable(15, "Alice", 95, "1.2s", "OK");
    printTable(15, "Bob", 87, "1.5s", "OK");
    return 0;}
```

### Your Output

Таблица с фиксированной шириной 15

|  | Name  | Score | Time | Status |
|--|-------|-------|------|--------|
|  | Alice | 95    | 1.2s | OK     |
|  | Bob   | 87    | 1.5s | OK     |

# Использование variadic и свёртки

## 10. Паттерн Visitor - паттерн проектирования позволяет добавлять новые операции к объектам, не изменяя их классы

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
// Применение разных операций к данным
template<typename T, typename... Visitors>
void applyAll(T& data, Visitors&&... visitors) {
    (std::forward<Visitors>(visitors)(data), ...);
}
// Пример 1: Работа с вектором чисел
void example1() {
    std::cout << "=== Example 1: Vector operations ===\n";
    std::vector<int> numbers = {5, 2, 3, 1, 4, 2, 3, 5, 1};

    std::cout << "Original: ";
    for(int n : numbers) std::cout << n << " ";
    std::cout << std::endl;
```

```
    applyAll(numbers,
        [](auto& v) {
            std::sort(v.begin(), v.end());
            std::cout << "After sort: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        },
        [](auto& v) {
            auto it = std::unique(v.begin(), v.end());
            v.erase(it, v.end());
            std::cout << "After unique: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        },
        [](auto& v) {
            std::reverse(v.begin(), v.end());
            std::cout << "After reverse: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        });
}
```

# Использование variadic и свёртки

## 10. Паттерн Visitor

### // Пример 2: Работа со строкой

```
void example2() {
    std::cout << "\n=== Example 2: String operations ===\n";
    std::string text = " Hello World ! ";
    std::cout << "Original: '" << text << "'" << std::endl;
    applyAll(text,
        [](auto& s) {
            s.erase(0, s.find_first_not_of(" "));
            s.erase(s.find_last_not_of(" ") + 1);
            std::cout << "Trimmed: '" << s << "'" << std::endl;
        },
        [](auto& s) {
            for(char& c : s) c = std::toupper(c);
            std::cout << "Uppercase: '" << s << "'" << std::endl;
        },
        [](auto& s) {
            std::reverse(s.begin(), s.end());
            std::cout << "Reversed: '" << s << "'" << std::endl;
        });
};
```

### // Пример 3: Фильтрация и трансформация

```
void example3() {
    std::cout << "\n=== Example 3: Filter and transform ===\n";
    std::vector<int> data = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    std::cout << "Original: ";
    for(int n : data) std::cout << n << " ";
    std::cout << std::endl;
    applyAll(data,
        [](auto& v) {
            v.erase(std::remove_if(v.begin(), v.end(),
                [](int x) { return x % 2 == 0; }), v.end());
            std::cout << "Only odds: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        },
        [](auto& v) {
            for(auto& n : v) n *= n;
            std::cout << "Squared: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        });
};
```

# Использование variadic и свёртки

## 10. Паттерн Visitor

### // Пример 2: Работа со строкой

```
void example2() {
    std::cout << "\n=== Example 2: String operations ===\n";
    std::string text = " Hello World ! ";
    std::cout << "Original: '" << text << "'" << std::endl;
    applyAll(text,
        [](auto& s) {
            s.erase(0, s.find_first_not_of(" "));
            s.erase(s.find_last_not_of(" ") + 1);
            std::cout << "Trimmed: '" << s << "'" << std::endl;
        },
        [](auto& s) {
            for(char& c : s) c = std::toupper(c);
            std::cout << "Uppercase: '" << s << "'" << std::endl;
        },
        [](auto& s) {
            std::reverse(s.begin(), s.end());
            std::cout << "Reversed: '" << s << "'" << std::endl;
        });
}
```

### // Пример 3: Фильтрация и трансформация

```
void example3() {
    std::cout << "\n=== Example 3: Filter and transform ===\n";
    std::vector<int> data = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    std::cout << "Original: ";
    for(int n : data) std::cout << n << " ";
    std::cout << std::endl;
    applyAll(data,
        [](auto& v) {
            v.erase(std::remove_if(v.begin(), v.end(),
                [](int x) { return x % 2 == 0; }), v.end());
            std::cout << "Only odds: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        },
        [](auto& v) {
            for(auto& n : v) n *= n;
            std::cout << "Squared: ";
            for(int n : v) std::cout << n << " ";
            std::cout << std::endl;
        });
}
```



# Использование variadic и свёртки

## 10. Паттерн Visitor

**// Пример 4: Разные типы данных**

```
void example4() {  
    std::cout << "\n=== Example 4: Multiple data types ===\n";  
    // Вектор строк  
    std::vector<std::string> words = {"apple", "banana", "cherry"};  
    applyAll(words,  
        [](auto& v) {  
            for(auto& w : v) w[0] = std::toupper(w[0]);  
            std::cout << "Capitalized: ";  
            for(const auto& w : v) std::cout << w << " ";  
            std::cout << std::endl;  
        },  
        [](auto& v) {  
            std::sort(v.begin(), v.end());  
            std::cout << "Sorted: ";  
            for(const auto& w : v) std::cout << w << " ";  
            std::cout << std::endl;  
        });  
}
```

```
int main() {  
    example1();  
    example2();  
    example3();  
    example4();  
    return 0;  
}
```

Example 1: Vector operations

Original: 5 2 3 1 4 2 3 5 1

After sort: 1 1 2 2 3 3 4 5 5

After unique: 1 2 3 4 5

After reverse: 5 4 3 2 1

Example 2: String operations

Original: ' Hello World ! '

Trimmed: 'Hello World !'

Uppercase: 'HELLO WORLD !'

Reversed: '! DLROW OLLEH'

Example 3: Filter and transform

Original: 1 2 3 4 5 6 7 8 9 10

Only odds: 1 3 5 7 9

Squared: 1 9 25 49 81

Example 4: Multiple data types

Capitalized: Apple Banana Cherry

Sorted: Apple Banana Cherry

# Использование variadic и свёртки

**11. Пакетное преобразование (Batch Transformation)** — применение одной и той же функции ко всем элементам набора данных с сохранением результатов в том же порядке

```
#include <iostream>
#include <tuple>
#include <string>
template<typename F, typename... Args>
auto applyToAll(F&& f, Args&&... args) {
    return std::make_tuple(f(std::forward<Args>(args))...);}
// Вспомогательная функция для печати tuple
template<typename Tuple, size_t... Is>
void printTupleImpl(const Tuple& t,
    std::index_sequence<Is...>) {
    ((std::cout << (Is == 0 ? "" : ", ") << std::get<Is>(t)), ...);}
template<typename... Args>
void printTuple(const std::tuple<Args...>& t) {
    std::cout << "(";
    printTupleImpl(t, std::index_sequence_for<Args...>{});
    std::cout << ")" << std::endl;}
```

```
int main() {
    // Пример 1: Возведение в квадрат
    auto result1 = applyToAll(
        [](int x) { return x * x; },
        1, 2, 3, 4, 5);
    std::cout << "Squares: ";
    printTuple(result1); // (1, 4, 9, 16, 25)
    // Пример 2: Преобразование в строку
    auto result2 = applyToAll(
        [](auto x) { return std::to_string(x); },
        42, 3.14, true, 'A');
    std::cout << "As strings: ";
    printTuple(result2); // (42, 3.140000, 1, A)
    return 0;}
```

## Your Output

```
Squares: (1, 4, 9, 16, 25)
As strings: (42, 3.140000, 1, 65)
```

# Использование variadic и свёртки

## 12. Парсинг команд - разбор аргументов командной строки

**Вход:** `./myprogram --host localhost --port 8080 --debug`

```
#include <iostream>
#include <map>
#include <string>
#include <vector>
template<typename... Args>
std::map<std::string, std::string> parseArgs(Args... args)
{
    std::map<std::string, std::string> result;
    std::vector<std::string> vec = {args...};
    for (size_t i = 0; i < vec.size(); i += 2) {
        if (i + 1 < vec.size())
            result[vec[i]] = vec[i + 1];
    }
    return result;
}
```

```
int main() {
    auto config = parseArgs("--host", "localhost",
"--port", "8080", "--debug", "true");
    for (const auto& [key, value] : config) {
        std::cout << key << " = " << value <<
std::endl;
    }
    return 0;
}
```

### Your Output

```
--debug = true
--host = localhost
--port = 8080
```

# Часть 2

**SFINAE и enable\_if**

## SFINAE и `enable_if`: Примеры использования

- **SFINAE** (Substitution Failure Is Not An Error) - правило C++:

если подстановка шаблонного параметра приводит к ошибке, это не считается ошибкой компиляции, а просто исключает эту перегрузку из рассмотрения

- **`enable_if`** - утилита из `<type_traits>`, которая использует SFINAE для включения/выключения перегрузок

**Обе функции имеют одинаковую сигнатуру** — возникает конфликт прегрузок. Например, при вызове **half(10)**, компилятор не знает какую функцию выбирать

#### Output

Status : Compilation error

#### Error

```
sol.cpp:14:22: error: call of overloaded 'half(int)' is ambiguous
```

```
14 |     std::cout << half(10) << std::endl;
```

```
|
```

```
~~~~~^~~~~
```

```
sol.cpp:4:5: note: candidate: 'int half(T) [with T = int]'
```

```
4 | int half(T x) {
```

```
|
```

```
~~~~~
```

```
sol.cpp:9:8: note: candidate: 'double half(T) [with T = int]'
```

```
9 | double half(T x) {
```

```
|
```

```
~~~~~
```

#### Без SFINAE

```
#include <iostream>
```

```
template<typename T>
```

```
int half(T x) {
```

```
    return x / 2;
```

```
}
```

```
template<typename T>
```

```
double half(T x) {
```

```
    return x / 2.0;
```

```
}
```

```
int main() {
```

```
    std::cout << half(10) << std::endl;
```

```
    return 0;
```

```
}
```

## C SFINAE

```
#include <iostream>
#include <type_traits>
// Целые числа
template<typename T>
std::enable_if_t<std::is_integral_v<T>, int>
half(T x) {
    std::cout << "[Integral] ";
    return x / 2;}
// Числа с плавающей точкой
template<typename T>
std::enable_if_t<std::is_floating_point_v<T>,
double>
half(T x) {
    std::cout << "[Floating] ";
    return x / 2.0;}
int main() {
    std::cout << half(10) << std::endl; // int
    std::cout << half(3.14) << std::endl; // double
}
```

Для активации SFINAE используем механизм:

`std::enable_if_t<условие, T> // C++14`

`std::enable_if<условие, T>::type // C++11`

### Your Output

```
[Integral] 5
[Floating] 1.57
```

## Механизмы SFINAE

| Механизм            | Минимальная версия                                                                                                                                                                                               |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| typename T::type    | C++98                                                                                                                                                                                                            |
| sizeof с выражением | C++98                                                                                                                                                                                                            |
| decltype            | C++11                                                                                                                                                                                                            |
| enable_if           | C++11                                                                                                                                                                                                            |
| enable_if_t         | C++14                                                                                                                                                                                                            |
| void_t              | C++17                                                                                                                                                                                                            |
| is_detected         | C++17 (экспериментальный*)<br>*is_detected — это утилита для <b>проверки валидности выражения</b> во время компиляции. Она была предложена для C++17, но <b>не вошла в стандарт</b> (осталась экспериментальной) |
| if constexpr        | C++17 (альтернатива SFINAE)                                                                                                                                                                                      |
| requires / Concepts | C++20                                                                                                                                                                                                            |



```
// Функция 1. Целые числа
template<typename T>
std::enable_if_t<std::is_integral_v<T>, int>
process(T x) {
    std::cout << "[Integral] ";
    return x / 2;}

// Функция 2. Числа с плавающей точкой
template<typename T>
std::enable_if_t<std::is_floating_point_v<T>, double>
process(T x) {
    std::cout << "[Floating] ";
    return x / 2.0;}

int main() {
    std::cout << process(10) << std::endl; // int
}
```

Your Output

[Integral] 5

Механизм создает ошибку, а SFINAE ее перехватывает:

**`std::enable_if_t<std::is_integral_v<T>>`**

`process(T x) { }`

Пример для вызова process(10) (T=int)

`std::enable_if_t<std::is_integral_v<int>>`

`std::enable_if_t<true> // is_integral_v<int> = true`

=> функция создаётся

`std::enable_if_t<std::is_floating_point_v<int>>`

`std::enable_if_t<false> // is_floating_point_v<int> = false`

=> функция не создаётся

SFINAE исключает функцию 2 из рассмотрения

## Concepts — механизм C++20, который позволяет напрямую указывать требования к шаблонным типам

**//C++14**

```
template<typename T>
std::enable_if_t<std::is_integral_v<T>, int>
half(T x) {
    return x / 2;
}
```

```
template<typename T>
std::enable_if_t<std::is_floating_point_v<T>,
double>
half(T x) {
    return x / 2.0;
}
```

**//C++20**

```
template<std::integral T>
int half(T x) {
    return x / 2;
}
```

```
template<std::floating_point T>
double half(T x) {
    return x / 2.0;
}
```

```
#include <iostream>
#include <concepts>
#include <string>
// Свой концепт
template<typename T>
concept has_speak = requires(T x) {
    { x.speak() } -> std::same_as<std::string>;
};
// Функция для целых чисел
template<std::integral T>
void process(T x) {
    std::cout << "Integral: " << x << std::endl;
}
// Функция для типов с методом speak
template<has_speak T>
void process(const T& x) {
    std::cout << "Has speak: " << x.speak() << std::endl;
}
struct Number {
    int value;
    std::string speak() const { return std::to_string(value); }
};
int main() {
    process(10);
    process(Number{5});
}
```

## Concepts C++20

### Your Output

```
Integral: 10
Has speak: 5
```

# enable\_if: Пример 1

```
#include <iostream>
#include <type_traits>
// Включить только для целых типов
template<typename T>
typename std::enable_if<std::is_integral<T>::value, T>::type
half(T value) {
    return value / 2;
}
// Включить только для типов с плавающей точкой
template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, T>::type
half(T value) {
    return value / 2.0;
}
int main() {
    std::cout << half(10) << std::endl;    // 5 (целая версия)
    std::cout << half(10.5) << std::endl;  // 5.25 (float версия)
    // half("text"); // Ошибка - нет подходящей перегрузки
}
```

## Your Output

5

5.25

## enable\_if: Пример 2 (C++14/17)

```
#include <iostream>
#include <type_traits>

template<typename T>
std::enable_if_t<std::is_integral_v<T>, T>
half(T value) {
    return value / 2;
}
// Альтернативный синтаксис с trailing return type
template<typename T>
auto half(T value) -> std::enable_if_t<std::is_floating_point_v<T>, T> {
    return value / 2.0;
}

int main() {
    std::cout << half(10) << std::endl;    // 5 (целая версия)
    std::cout << half(10.5) << std::endl;  // 5.25 (float версия)
    std::cout << half(7.0) << std::endl;   // 3.5
    std::cout << half(15) << std::endl;    // 7
    return 0;
}
```

### Your Output

5

5.25

3.5

7

## Пример 3. Использование с контейнерами

// Без SFINAE перегрузка нужна для каждого типа

```
template<typename T>
```

```
T getMiddle(std::vector<T>& v) { /* ... */ }
```

```
template<typename T>
```

```
T getMiddle(std::deque<T>& d) { /* ... */ }
```

```
template<typename T>
```

```
T getMiddle(std::array<T, N>& a) { /* ... */ }
```

```
template<typename T>
```

```
T getMiddle(std::valarray<T>& va) { /* ... */ }
```

// С SFINAE нужна одна перегрузка для BCeX random\_access контейнеров

```
template<typename Container>
```

```
auto getMiddle(Container& cont) ->
```

```
    std::enable_if_t<is_random_access<Container>,  
                    typename Container::value_type>
```

```
{
```

```
    return cont[cont.size() / 2]; // Работает для vector, deque, array, valarray...
```

```
}
```

## Пример 4. Проверка наличия метода

```
#include <iostream>
#include <vector>
#include <list>    // ← Нужно добавить этот заголовок
#include <type_traits> // для std::void_t, std::declval
// Проверка наличия метода reserve
template<typename T, typename = void>
struct has_reserve : std::false_type {};
template<typename T>
struct has_reserve<T, std::void_t<decltype(std::declval<T>().reserve(0))>>
    : std::true_type {};
// Версия для контейнеров с reserve
template<typename Container>
std::enable_if_t<has_reserve<Container>::value>
optimize(Container& cont, size_t size) {
    cont.reserve(size);
    std::cout << "Using reserve optimization\n";
}
// Версия для контейнеров без reserve
template<typename Container>
std::enable_if_t<!has_reserve<Container>::value>
optimize(Container& cont, size_t size) {
    std::cout << "No reserve available\n";
}
```

```
int main() {
    std::vector<int> vec;
    // Using reserve optimization
    optimize(vec, 100);
    // list определён
    std::list<int> lst;
    // No reserve available
    optimize(lst, 100);
    return 0;
}
```

### Your Output

```
Using reserve optimization
No reserve available
```

## Пример 5. Разные типы аргументов

```
#include <iostream>
#include <type_traits>

// Функция работает только если оба аргумента одного типа
template<typename T, typename U>
auto add(T a, U b) ->
    std::enable_if_t<std::is_same_v<T, U>, T>
{
    return a + b;
}

// Перегрузка для разных типов - конвертируем к общему
template<typename T, typename U>
auto add(T a, U b) ->
    std::enable_if_t<!std::is_same_v<T, U>, decltype(a + b)>
{
    std::cout << "Converting types: " << typeid(T).name()
        << " + " << typeid(U).name() << " = ";
    return a + b;
}

int main() {
    std::cout << add(5, 10) << std::endl;    // 15 (один тип)
    std::cout << add(5.5, 10) << std::endl;  // Converting types: double + int =
15.5
}
```

### Your Output

15

Converting types: d + i = 15.5



## Пример 6. Использование `enable_if` с классами

```
#include <iostream>
#include <type_traits>
template<typename T, typename Enable = void>
class Processor;
// Специализация для чисел
template<typename T>
class Processor<T,
std::enable_if_t<std::is_arithmetic_v<T>>>
{public:
    T process(T value) {
        return value * 2;
    }
};
// Специализация для строк
template<>
class Processor<std::string> {
public:
    std::string process(const std::string& value) {
        return "\"" + value + "\"";
    }
};
```

```
int main() {
    Processor<int> intProc;
    std::cout << intProc.process(21) <<
std::endl;
    Processor<double> doubleProc;
    std::cout << doubleProc.process(3.14) <<
std::endl;
    Processor<std::string> strProc;
    std::cout << strProc.process("Hello") <<
std::endl;
}
```

### Your Output

42

6.28

"Hello"

## Пример 7. Использование enable\_if с классами

```
#include <iostream>
#include <type_traits>
template<typename T, typename Enable = void>
class Processor;
// Специализация для чисел
template<typename T>
class Processor<T, std::enable_if_t<std::is_arithmetic_v<T>>>
{
public:
    T process(T value) {
        return value * 2;
    }
};
// Специализация для строк
template<>
class Processor<std::string> {
public:
    std::string process(const std::string& value) {
        return "\"" + value + "\"";
    }
};
```

```
int main() {
    Processor<int> intProc;
    std::cout << intProc.process(21) << std::endl;
    Processor<double> doubleProc;
    std::cout << doubleProc.process(3.14) << std::endl;
    Processor<std::string> strProc;
    std::cout << strProc.process("Hello") << std::endl;
}
```

### Your Output

42

6.28

"Hello"

**SFINAE/enable\_if** используется, например, для:

- Реализации/запрета перегрузки на основе свойств типов
- Условий, когда нельзя использовать обычную перегрузку
- Проверки наличия метода/члена
- Работы со старыми компиляторами (до C++17/20)

В C++17/20 можно использовать `if constexpr/concepts`